Report

Project Web datamining & semantics

Clarisse Ballon & Léa Hadj-Said

DIA 1

1- Data Acquisition & Information Extraction

Domain and Seed URLs

We chose AI in architecture as the theme for this project. This topic sits at the intersection of computer science and design, two fields that are increasingly connected in practice. The goal was to build a knowledge base capturing entities and relationships related to how AI is used in architectural design, generative design, building information modeling (BIM) and urban planning.

We started with eight academic articles from ScienceDirect, Springer and MDPI. However, three URLs failed because these platforms use strict anti-scraping protections. We had to replace them with more accessible sources with the website Wikipedia.

This gave us a final corpus of 13 usable articles which is on the low side.

Crawler Design and Ethics

The crawler uses two Python libraries: httpx for fetching raw HTML and trafilatura for extracting the main textual content. Trafilatura removes navigation menus, footers, ads and cookie banners from web pages, keeping only the article body. We also added a minimum word count filter of 500 words to keep only substantial pages.

The crawler respects standard ethics: it only fetches public pages, identifies itself with a proper User-Agent header and does not crawl at high speed.

Cleaning Pipeline

Each document is saved in JSONL format with its URL, extraction timestamp, word count and cleaned text. The pipeline steps are fetching HTML with httpx, extract main content with trafilatura after we filter documents with fewer than 500 words and finally, we save valid documents to crawler_output.jsonl.

Named Entity Recognition

For information extraction, we used spaCy with the en_core_web_trf transformer model, which is more accurate than the default small model. The NER step identified four types of entities: PERSON, ORG, GPE and DATE. These categories were chosen because they are most relevant to our domain people in architecture and AI, organizations, countries and cities, and dates.

The extraction produced 3499 raw triplets but only 1728 were unique after deduplication. The difference shows there is a lot of repetition across the articles. Some representative entity examples directly from the generates file.

extracted_knowledge

entity	label	source_url
1 April 2018	DATE	https://en.wikipedia.org/wiki/Smart_city
1 April 2019	DATE	https://en.wikipedia.org/wiki/Simulation

Figure 1: Sample of extracted entities

Ambiguity Cases

During the next step we encountered three main ambiguity cases:

- "May": target as DATE (the mouth) and as PERSON. Without context disambiguation this caused incorrect entity labels.
- "Apple": sometimes tagged as ORG and sometimes missed entirely. The model did not always correctly identify it in architecture texts.
- "China": tagged as GPE but also as ORG especially when referring to government policies. This ambiguity was not resolved in our pipeline and introduced noise.

2- KB Construction & Alignment

RDF Modelling Choices

The knowledge base uses Wikidata as the reference ontology. All entities are represented as Wikidata URIs, and all relations use Wikidata direct properties. This choice was made because Wikidata is one of the largest open knowledge bases and provides a standardized vocabulary for entities and relations. Entities and relations were also converted to URIs using a camelCase normalization function.

Entity Linking with Confidence

We originally planned to use DBpedia Spotlight, but the conversion from DBpedia URIs to Wikidata QIDs failed completely. We therefore replaced this step with direct calls to the Wikidata Search API, which returns QIDs directly.

A quality filter was applied to remove false positives. After filtering, 253 entities out of 507 candidates were successfully linked to Wikidata QIDs.

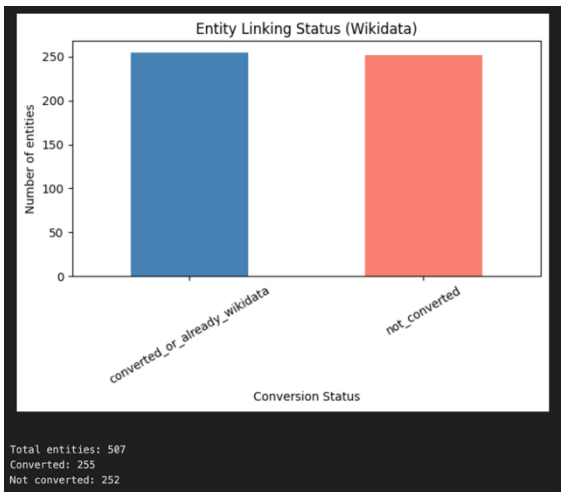

Figure 2: Number of entities linking with Wikidata

Predicate Alignment

For predicate alignment, we used SPARQL queries on Wikidata to find official relations between entity pairs where both had valid QIDs. Out of 807 triplets, 159 had both entities aligned to Wikidata. Among those, only 8 predicate candidates were found. This very low number shows that most of our extracted relations (shape, showcase, clash...) do not correspond to official Wikidata properties.

The 8 candidates included properties like P17 (country), P159 (headquarters location) and P1830 (owner of). These relations were used as reference for the expansion step.

Expansion Strategy

To reach the required KB size of 50 000-200 000 triplets, we expanded the KB using SPARQL queries on Wikidata with four complementary strategies:

- 1-hop expansion: for each of the 253 core entities, retrieve all direct Wikidata properties filtered on 51 controlled predicates (LIMIT 500 per entity)
- Predicate-controlled expansion: retrieve triples with specific predicates for all core entities
- 2-hop expansion: navigate through intermediate entities via 9 key predicates, also filtered on the controlled list
- Second pass: apply predicate-controlled expansion to newly discovered entities

The 51 predicates were manually selected to stay relevant to the AI in architecture domain. They cover organizational relations (P108 employer, P463 member of), creative relations (P170 creator, P84 architect), classification relations (P31 instance of, P279 subclass of) and domain-specific ones (P277 programmed in, P921 main subject).

Final KB Statistics

Metric	Value	Requirement	Status
Total triplets	42,326	50 000 - 200 000	Below target
Total entities	29,758	5 000 - 30 000	OK
Total relations	344	50 - 200	Above target

The KB did not fully meet the requirements. The number of relations (344) was higher than the 200 maximum because the 1-hop expansion retrieved predicates outside the controlled list. The triplet count was slightly below the 50 000 minimum. We are near to have good requirement, so we keep these statistics.

3- Reasoning with SWRL

For rule-based reasoning, we worked with a predefined family ontology (family.owl) using the owlready2 library and the Pellet reasoner. The ontology contains classes like Person, Male, Female, Child, Parent and Grandparents with individuals like Peter, Marie, Marc and Anna.

We defined the following SWRL rule to automatically classify people over 60 as old.

```
SWRL rule added:
Person(?p), age(?p, ?a), greaterThan(?a, 60) -> oldPerson(?p)
```

Figure 4: New rule for classify people over 60 years old

After running the Pellet reasoner, Peter and Marie were automatically classified as oldPerson.

```
Individuals inferred as oldPerson:
- Peter
- Marie
```

Figure 3: Individuals Inferred as oldPerson

The reasoner also inferred new class memberships from the ontology structure:

- Daughter is both Child and Female
- Son is both Child and Male
- Grandmother is both Mother and Grandparents
- Uncle is a Brother

This shows that rule-based reasoning is very powerful for well-defined ontologies once the rules are set, the reasoner automatically derives all consequences without any additional code.

```
Running reasoner...
* Owlready2 * Running Pellet...
    java -Xmx2000M -cp /Users/clarisseballon/Library/CloudStorage/OneDrive-Personnel/Documents/ESILV/A4/S8/Web
Reasoning finished.
* Owlready2 * Pellet took 0.7355861663818359 seconds
* Owlready * Reparenting family.Daughter: {family.Child} => {family.Child, family.Female}
* Owlready * Reparenting family.Son: {family.Child} => {family.Child, family.Male}
* Owlready * Reparenting family.Grandmother: {family.Female, family.Grandparents} => {family.Mother, family.Gr
* Owlready * Reparenting family.Grandfather: {family.Male, family.Grandparents} => {family.Father, family.Gran
* Owlready * Reparenting family.Uncle: {family.Male} => {family.Brother}
* Owlready * (NB: only changes on entities loaded in Python are shown, other changes are done but not listed)
```

Figure 5: Reasoner inferred new class memberships

SWRL Rule Concept for Our KB

Although our KB is based on Wikidata URIs and does not have a custom OWL ontology, the same principle could be applied. For example, a rule like the following could automatically mark relevant entities: Entity=>instance_of(ArchitectureSoftware)=>relevant(AIArchitecture).

This rule would classify any entity that is an instance of architecture software as relevant to our domain. In practice this was not implemented on the Wikidata KB because building a full OWL ontology over 29,000 entities was beyond the scope of this project.

4- Knowledge Graph Embeddings

Data Preparation and Splits

Before training, the KB was cleaned: duplicates removed, invalid URIs filtered, literal objects removed, and the largest connected component kept. The final KB contained 42 326 triplets with 29 758 entities and 344 relations. Entities and relations were then converted to numerical IDs because KGE models cannot process string values.

The data was split 80/10/10 (train/valid/test) with the constraint that all entities in validation and test must also appear in training. This is necessary because KGE models cannot make predictions for unseen entities.

```
SPLIT STATISTICS
Total triples : 42326
Train triples : 38993 (92.13%)
Valid triples : 1598 (3.78%)
Test triples  : 1735 (4.10%)
```

Figure 6: Split to train - valid - test

Training Configuration

Both models were trained with the same hyperparameters:

Parameter	Value	Justification
Embedding dimension	100	Good balance between expressiveness and speed
Learning rate	0.001	Stable convergence with Adam optimizer
Batch size	256	Balance between noise and memory usage
Epochs	100	Standard value for this KB size
Negative sampler	PseudoTyped	More challenging negatives, better accuracy

TransE Model

The TransE model learns by comparing true and corrupted triples. It adjusts the embeddings so that valid triples are close in the vector space while invalid triples are far apart.

ComplEx Model

ComplEx is an embedding model that represents entities and relations in a complex vector space. Unlike TransE it can model asymmetric relations meaning that it can distinguish between (h, r, t) and (t, r, h). This makes it more expressive and better suited for real-world knowledge graphs.

Comparison of the two model with Metrics: MRR, Hits@1/3/10

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.043	0.003	0.069	0.105
ComplEx	0.000236	0.000	0.000	0.000289

TransE significantly outperforms ComplEx on all metrics. An MRR of 0.043 means the correct answer is found around the 23rd position on average. While this is low, it shows TransE learned something from the data. ComplEx essentially failed to learn, its MRR is nearly zero. Although ComplEx is theoretically more expressive, our KB with 344 sparse relations made it very difficult for ComplEx to converge.

Size Sensitivity Analysis

We compared both models trained on 20k triplets vs the full dataset. We did not include a 50k case because our KB was close to that size and the results would have been too similar.

Model	Dataset	Train Triples	MRR	Hits@10
TransE	20k	16,000	0.019	0.046
TransE	Full	33,860	0.046	0.112
ComplEx	20k	16,000	0.000376	0.000
ComplEx	Full	33,860	0.000612	0.001

For TransE, more data clearly improves performance, MRR nearly doubles going from 20k to the full dataset. ComplEx remains unable to learn regardless of dataset size, confirming the problem is structural rather than a lack of data.

Nearest Neighbours

After training TransE, we extracted entity embeddings and computed nearest neighbors using cosine similarity. The results were visualized using PCA reduction to 2D. Architecture software entities like Autodesk and ArchiCAD are positioned close to each other in the embedding space, and tech companies like Google and Alan Turing form another group. Countries like France and Germany cluster together near the center.

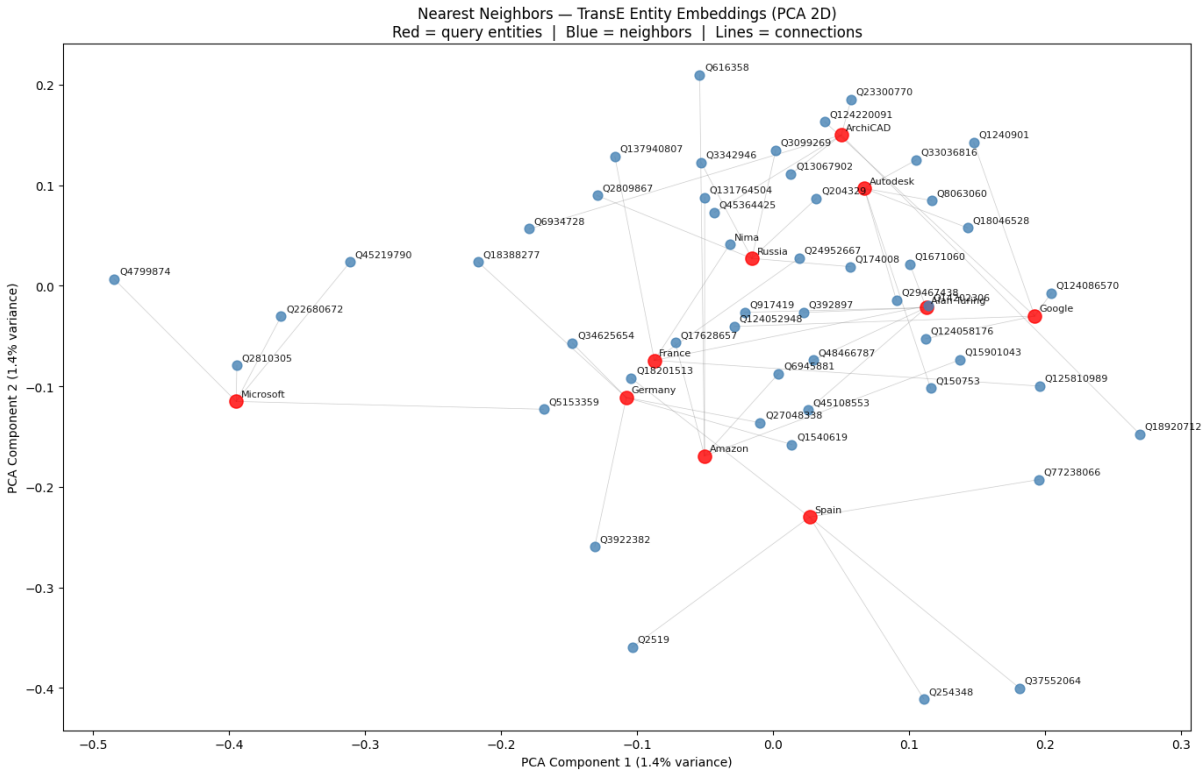

Figure 7 :Nearest Neighbors

Clustering Analysis

The t-SNE visualization shows how TransE organized the knowledge graph entities in 2D space. The entities are relatively spread out without sharp boundaries, which is typical for densely interconnected knowledge graphs. Geographic entities like Hong Kong, Taipei, Malaysia, Cambridge and Amsterdam tend to appear on the left side. Technology companies and media organizations like Apple, Netflix, Airbnb and ESPN cluster in the upper-center area. Academic institutions like Harvard and the University of Sydney appear toward the bottom-center.

However, the graph also reveals noise: entities like 'Santa Claus', 'Skywalker' and isolated dates appear scattered, suggesting the KB contains heterogeneous entities the model struggled to place consistently.

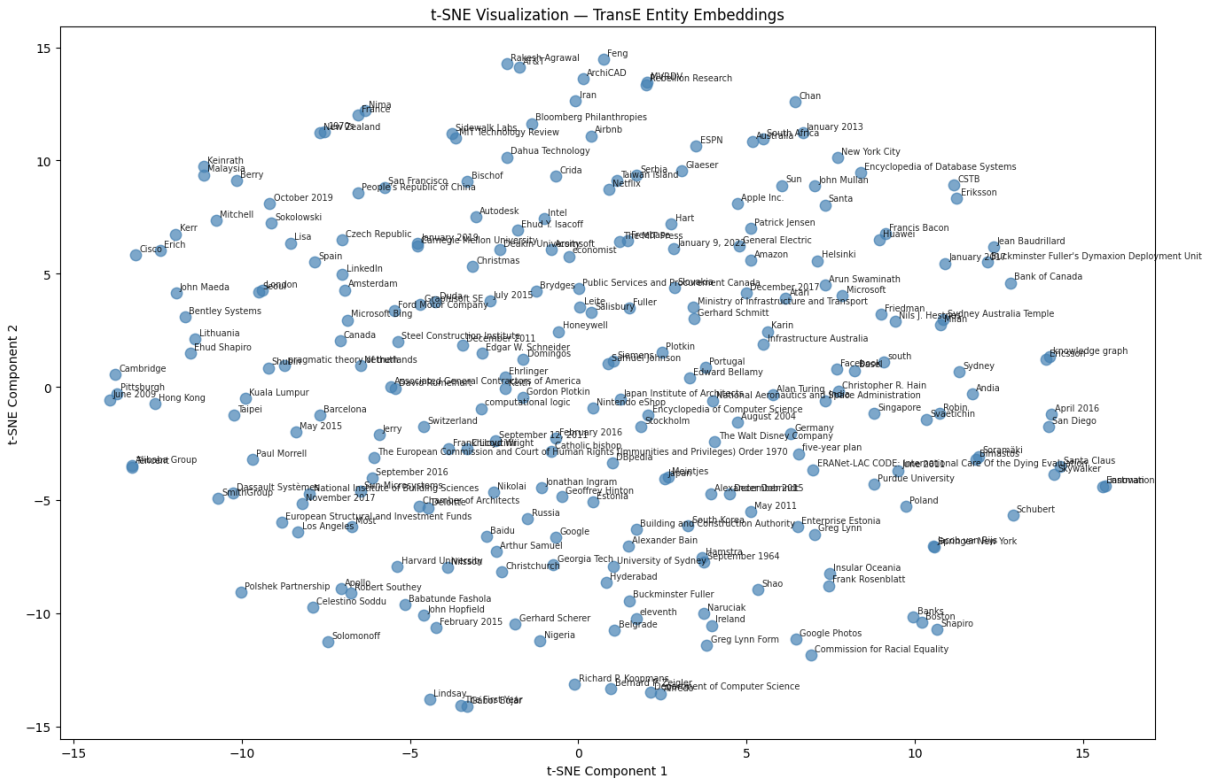

Figure 8: t-SNE Visualization

We then applied K-Means clustering with k=3 (chosen by silhouette score = 0.0125). Three clusters were identified: Cluster 1 (red) contains academic and architecture organizations like Graphisoft SE, ArchiCAD and Carnegie Mellon University. Cluster 2 (blue) groups major tech companies and geographical entities. Cluster 3 (green) contains specialized entities like MVRDV and Sidewalk Labs.

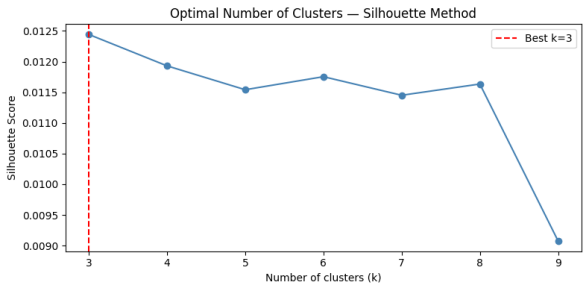

Figure 9: Number of clustering

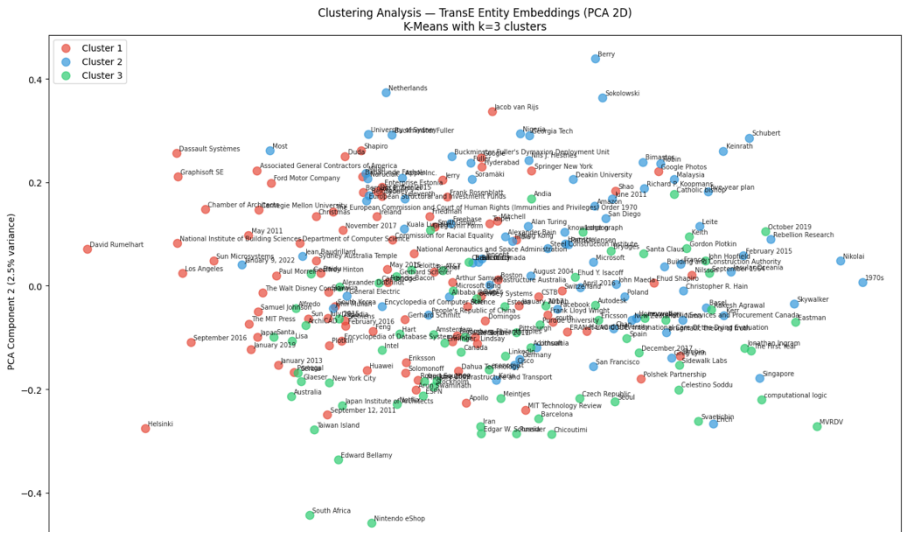

Figure 10: Clustering Analysis

Relation Behaviour

The most frequent relations in the KB are “has part” with 2939 triples, “instance of” with 2937 triples and “main subject” with 2675 triples. These are typical Wikidata structural relations. Domain-specific relations like “employer” with 1589, “founded at” with 1725 and “named after” with 1771 also appear prominently.

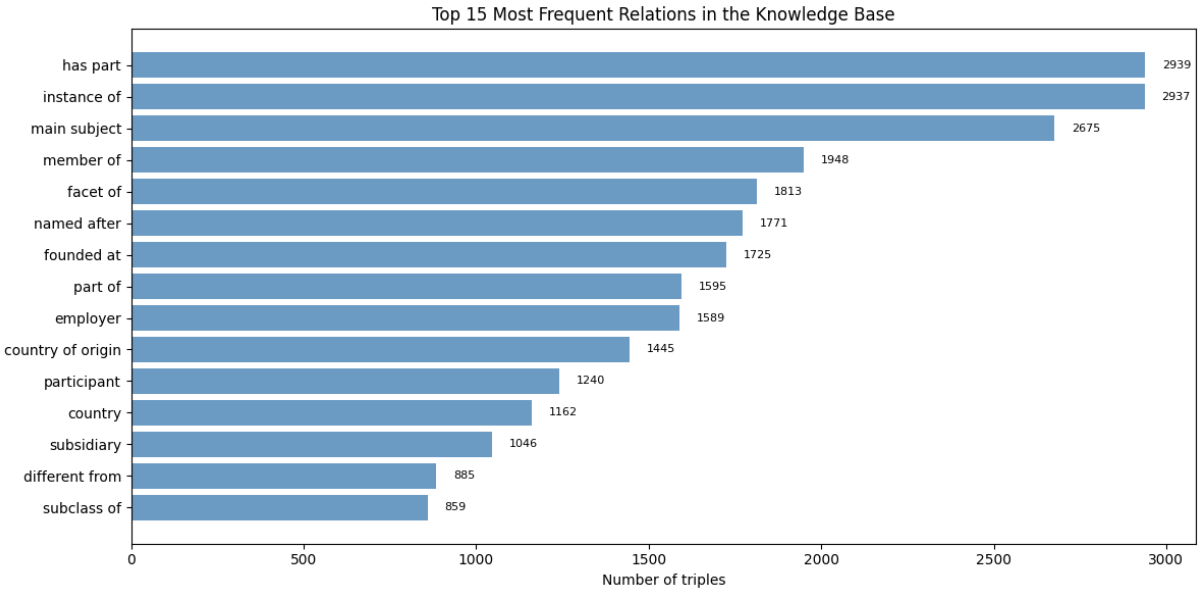

Figure 11: Most Frequent Relation in the KB

5- RAG over RDF/SPARQL

Schema Summary

A schema summary is built automatically from the RDF graph by extracting prefixes, distinct predicates and sample triples. This summary gives the LLM the context it needs to generate valid SPARQL. It includes the wd: and wdt: prefixes, 11 key predicates with human-readable names (P31 = instance of, P108 = employer, P84 = architect...) and 20 sample triples.

NL to SPARQL Prompt Template

The prompt is structured as: SPARQL instructions, then the schema summary, then the natural language question. The LLM is instructed to return only a valid SPARQL SELECT query in a code block. A validation function also automatically adds missing PREFIX declarations and fixes common SELECT clause errors.

Self-Repair Mechanism

If the generated SPARQL fails to execute, the system sends the broken query along with the error message and schema summary back to the LLM with a repair prompt. The LLM generates a corrected version. This self-repair mechanism was triggered once during testing when the first generated query had an invalid SELECT clause.

Evaluation with 5 Questions

We evaluated our RAG system on five questions covering different aspects of our AI in Architecture knowledge base.

Question	Baseline (LLM only)	RAG (SPARQL + KB)	Result
What is Microsoft an instance of?	Microsoft is an instance of software and technology, specifically focusing on operating systems, productivity software, and cloud services.	Entity: Calendar month	Partial RAG executed but wrong entity
What is Q2283 (Microsoft) an instance of?	Q2283 is an instance of “Business Analysis” within the ServiceNow platform. It’s a workflow that helps you manage and track business process analysis.	Returns: business, organization, public company via P31, privately held company	RAG correct
Who is the creator of AutoCAD?	Robert F. Hayes created AutoCAD.	[No rows returned]	Baseline better data not in KB
What is BIM an instance of?	BIM stands for Building Information Modeling. It’s a digital representation of a physical or virtual structure, combining 3D data with information about its materials, components, and properties.	Returns: five-year plan, taxon	RAG incorrect noise from expansion
Which entities are influenced by Alan Turing?	Alan Turing significantly influenced numerous areas, including: • Computer Science: His work on the Turing machine and theoretical foundations of computers is crucial. • Cryptography: He played a key role in breaking Enigma codes during WWII. • Artificial Intelligence: His ideas about thinking machines and computation laid groundwork for AI research. • Philosophy: His work on computability and the potential for machines to think challenged philosophical assumptions. Essentially, he was a foundational figure in the development of modern computing and its implications.	Returns: Max Newman	Partial RAG returns one grounded result

When using explicit Wikidata QIDs, the RAG system correctly retrieved grounded answers from the knowledge base. For example, for the question "What is Q2283 an instance of?", the RAG returned business, organization and public company via P31, while the baseline LLM incorrectly described Q2283 as a ServiceNow workflow. This shows the main advantage of RAG, it retrieves verifiable facts instead of generating potentially wrong answers.

However, two failure cases were observed. First, when entity names are used instead of QIDs, the small LLM maps to incorrect Wikidata IDs, leading to wrong results. Second, for questions like "What is BIM an instance of?", the RAG returned irrelevant entities like taxon and five-year plan, which is caused by the noise introduced during the Wikidata expansion step.

Overall, the RAG approach works well when QIDs are explicitly provided but struggles with entity resolution from natural language. A larger LLM would significantly improve the results.

6- Critical Reflection

Impact of KB Quality on Embedding Performance

The main lesson from this project is that embedding performance depends completely on the quality of the upstream pipeline. We started with only 5 articles and the entities extracted by spaCy were often noisy or fragmentary and finally we have 13 articles. This directly limited Wikidata alignment (253 out of 507 entities) and the expansion quality. The final KB had 344 relations instead of the 200 maximums, making it much harder for the models to learn meaningful patterns since each relation needs enough examples to be properly learned.

Impact of Noisy Expansion

The KB expansion via SPARQL introduced a lot of noise. The 1-hop expansion retrieved all direct properties of each entity, which includes many unrelated to AI in architecture such as sports teams, geographic subdivisions and historical figures. This noise made the embedding space less coherent and harder to interpret. The presence of entities like 'Santa Claus' and “Skywalker” in the t-SNE visualization confirms this.

Effect of Ontology Modeling Choices

The 51 controlled predicates were chosen manually, which was somewhat arbitrary. Some relevant relations may have been excluded while irrelevant ones were included. A better approach would have been to use a domain-specific ontology for architecture or AI, which would have provided a more principled and reproducible set of relations.

Open-World vs Closed-World Assumption

KGE models like TransE and ComplEx assume a closed world, if a triple is absent from the KB, it is treated as false. But our KB was built under the open-world assumption where a missing triple might simply not have been collected rather than being false. This fundamental mismatch penalizes the models for predicting triples that might be true but are just not in our dataset.

Rule-Based vs Embedding-Based Reasoning

Both approaches are complementary rather than competing. Rule-based reasoning is ideal for well-defined domains like the family ontology where logical rules can be written explicitly. Embedding-based reasoning is better for large incomplete KBs where patterns need to be discovered automatically. In the context of AI in architecture, a hybrid approach combining both would likely produce the best results.

What we would improve

Looking back at the project, several improvements could significantly enhance the results.

The most impactful change would be to collect more articles at least 20 to 30 instead of our 13 to get a larger and more diverse corpus. This would directly improve the quality of the extracted entities and relations, and therefore everything that follows in the pipeline.

We would also use a NER model fine-tuned on architecture and AI domain text instead of the generic spaCy model, which would reduce the number of irrelevant entities extracted. On the KB side, filtering the Wikidata expansion more aggressively would help keep the number of relations below 200 and reduce noise like the taxon entities we observed in the RAG results.

For the embedding models, training for 200+ epochs with a GPU would give TransE more time to converge and likely improve the MRR significantly. Finally, using a larger LLM like Llama 3 8B for SPARQL generation would solve the entity resolution problem we encountered in the RAG evaluation, where the small model kept mapping entity names to wrong QIDs.

In the longer term, building a proper domain ontology for AI in architecture would replace the manual predicate selection and provide a more principled and reproducible knowledge base structure.